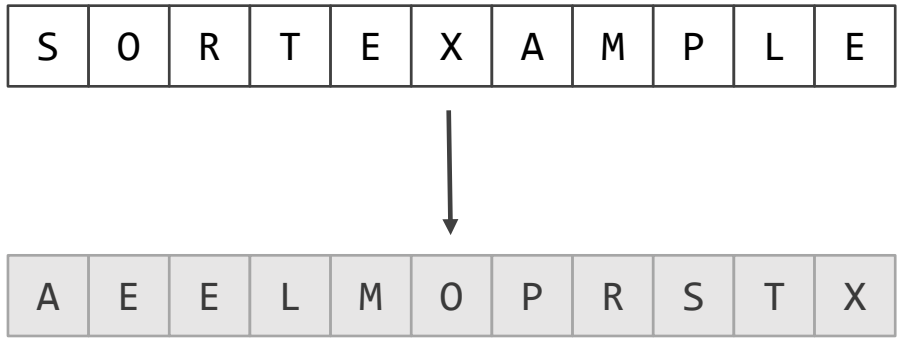
Elementary Sorts

Selection sort and insertion sort · Algorithms § 2.1

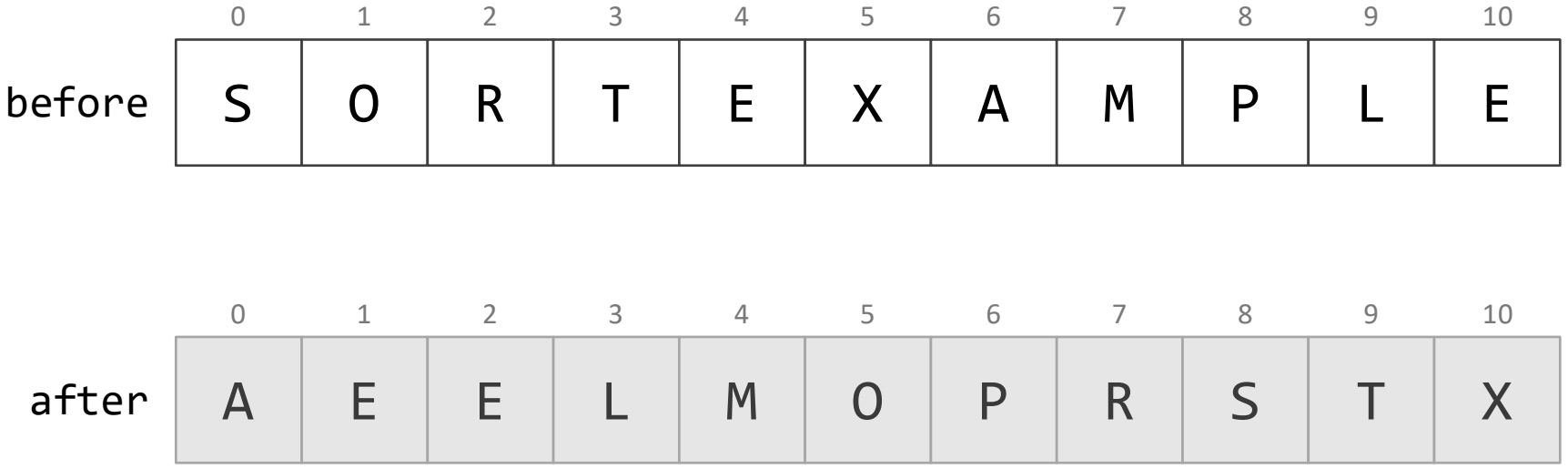
Agenda for this video

- The sorting problem, and the rules every sort in the book follows
- **Selection sort**: find the smallest, put it first, repeat
- **Insertion sort**: slide each item left into place
- What you need for part 1 of the next homework assignment 4



The sorting problem

Rearrange the n items in an array so they're in **ascending order**.



On slides we'll sort letters, because they're easy to read. The same code sorts numbers, names, or anything else with a natural order.

Why we care which sort we use

growth rate	**n = 1,000**	**n = 1,000,000**
n log n	10,000	20,000,000
n^2	1,000,000	1,000,000,000,000

Today's two sorts are both **n^2**.

Next video's sort, mergesort, is **n log n**.

From the big-O slides: if an n log n sort takes **one second** on a million items, an n^2 sort takes about **14 hours**.

Sorting any kind of item: Comparable

```
1  "A".compareTo("B")    // negative: "A" comes first
2  "B".compareTo("B")    // zero: they're equal
3  "C".compareTo("B")    // positive: "C" comes after
```

`v.compareTo(w)` returns a **negative** number, **zero**, or a **positive** number, depending on whether v comes before, is equal to, or comes after w.

`String`, `Integer`, and `Double` already implement `Comparable`. So our sorts take a `Comparable[]`, and the same method sorts letters or numbers.

The two helper methods: less and exch

```
1  // Is v < w?
2  private static boolean less(Comparable v, Comparable w) {
3      return v.compareTo(w) < 0;
4  }
5
6  // Exchanges a[i] and a[j].
7  private static void exch(Comparable[] a, int i, int j) {
8      Comparable t = a[i];
9      a[i] = a[j];
10     a[j] = t;
11 }
```

Every sort in the book looks at the array **only** through these two methods.

Every comparison is a call to `less`.

Every swap is a call to `exch`.

Both are already written in your starter code.

What we count: compares and exchanges

Same idea as the big-O slides: count the operation that runs most often, instead of timing.

we count	which means	in the code
compares	asking whether one item is less than another	`less(v, w)`
exchanges	swapping two items in the array	`exch(a, i, j)`

Because every sort goes through `less` and `exch`, we always know exactly what to count. When the book says a sort makes "~$n^2/2$ compares," this is what it's counting.

Checking the answer: isSorted

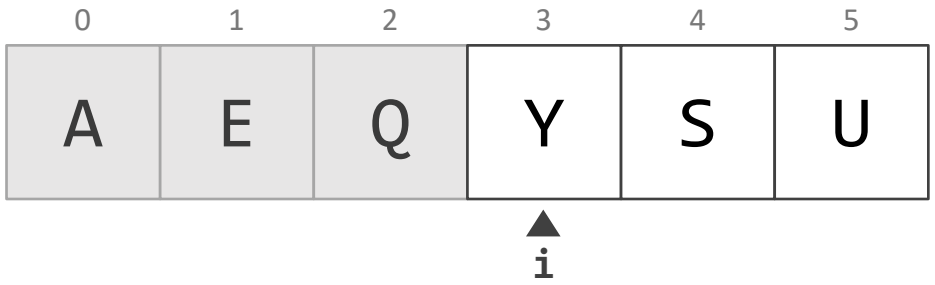
```
1  public static boolean isSorted(Comparable[] a) {
2      for (int i = 1; i < a.length; i++) {
3          if (less(a[i], a[i - 1])) {
4              return false;
5          }
6      }
7      return true;
8  }
```

If any item is less than the one before it, the array isn't sorted.

`SortClient` uses this to check your sorts. When it prints OK, your sort passed.

Selection sort: the idea

1. Find the **smallest** item in the array. Exchange it with `a[0]`.
2. Find the smallest of the **rest**. Exchange it with `a[1]`.
3. Keep going until you reach the end.

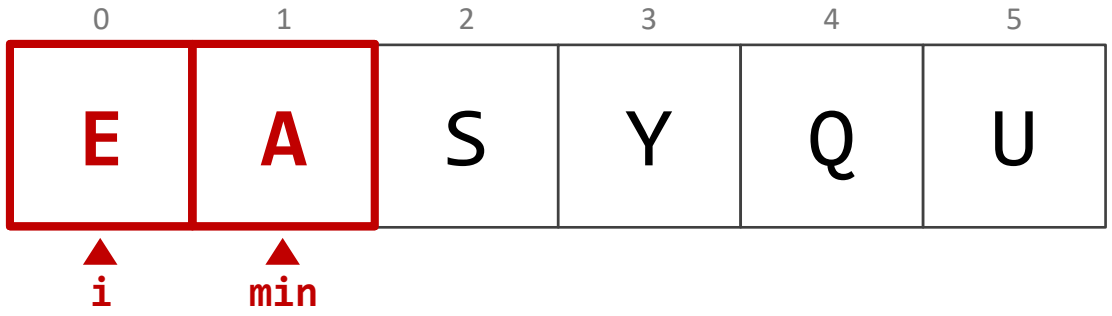


Each pass, the finished part on the left grows by one.

finished: will never move again

what's moving now

Selection sort: trace



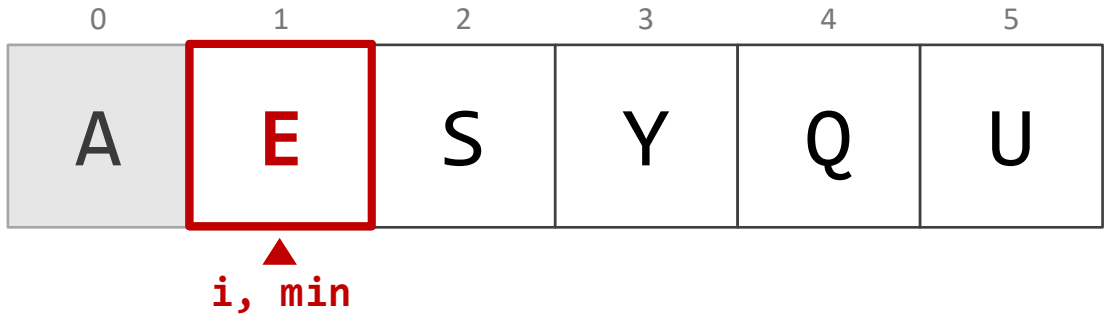
Pass i = 0

The smallest of a[0..5] is **A**, at index 1. Exchange a[0] and a[1].

finished

about to be exchanged

Selection sort: trace

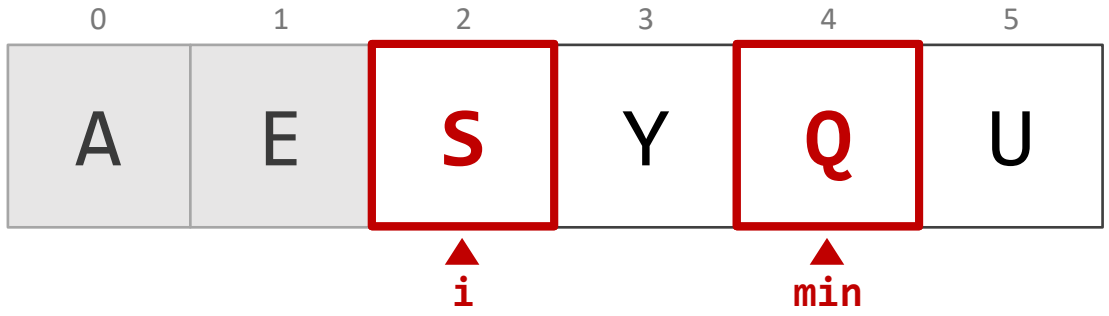


Pass i = 1

The smallest of a[1..5] is **E**, and it's already at a[1]. Exchanging it with itself changes nothing.

finished about to be exchanged

Selection sort: trace

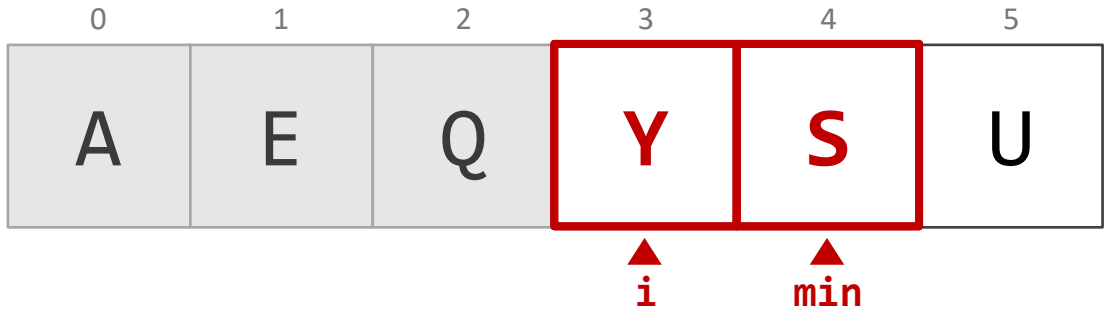


Pass i = 2

The smallest of a[2..5] is **Q**, at index 4. Exchange a[2] and a[4].

finished about to be exchanged

Selection sort: trace



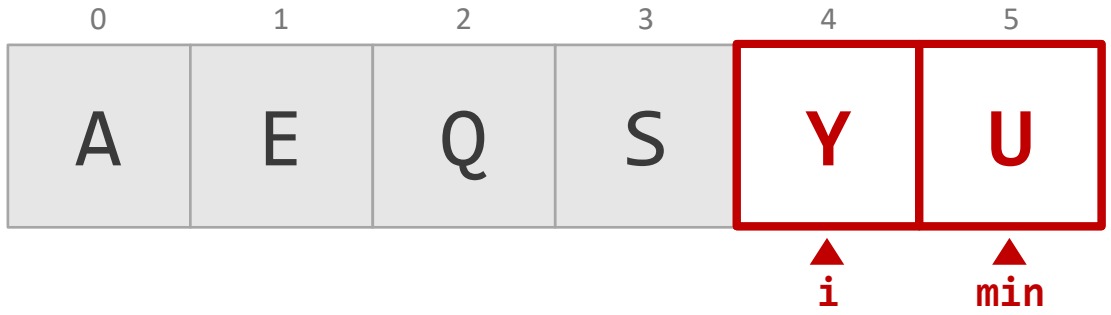
Pass i = 3

The smallest of a[3..5] is **S**, at index 4. Exchange a[3] and a[4].

finished

about to be exchanged

Selection sort: trace



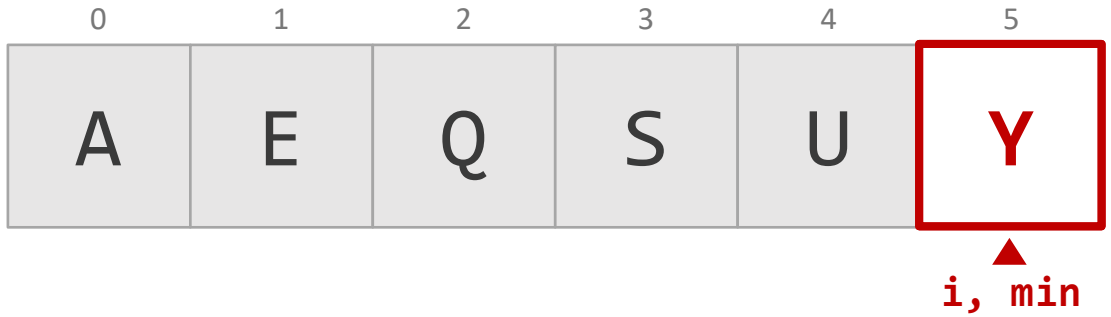
Pass i = 4

The smallest of a[4..5] is **U**, at index 5. Exchange a[4] and a[5].

finished

about to be exchanged

Selection sort: trace

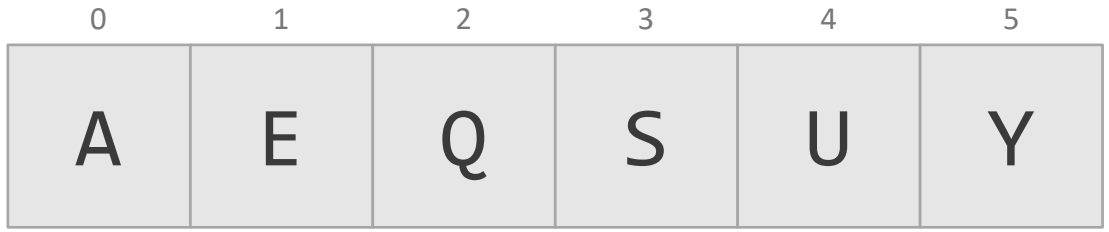


Pass i = 5

The smallest of a[5..5] is **Y**, and it's already at a[5]. Exchanging it with itself changes nothing.

finished about to be exchanged

Selection sort: trace



Done. Six passes, six exchanges (including the two that exchanged an item with itself).

Selection sort: the code

```
1  public static void sort(Comparable[] a) {
2      int n = a.length;
3      for (int i = 0; i < n; i++) {
4          int min = i;
5          for (int j = i + 1; j < n; j++) {
6              if (less(a[j], a[min])) {
7                  min = j;
8              }
9          }
10         exch(a, i, min);
11     }
12 }
```

This is `Selection.java`, and it's **complete** in your starter code.

Lines 5–9 scan the rest of the array for the smallest item.

Line 10 exchanges it into position.

Read it before you start: your sorts follow the same pattern.

Selection sort: how much work?

Pass 0 makes n − 1 compares, pass 1 makes n − 2, and so on down to 1:

(n − 1) + (n − 2) + … + 1 = n(n − 1)/2, which is **~$n^2/2$ compares**.

Exactly **n exchanges**: one per pass.

n	compares	exchanges
10	45	10
1,000	499,500	1,000
1,000,000	499,999,500,000	1,000,000

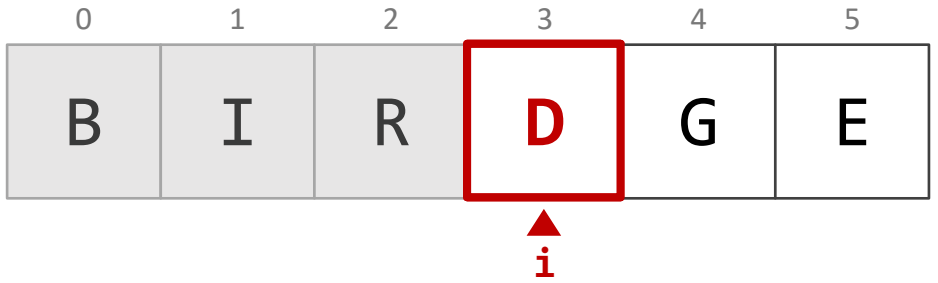
It can't tell when it's done.

Give selection sort an array that's *already sorted*, and it still makes ~$n^2/2$ compares. It has to scan the rest of the array to be sure it found the smallest.

Insertion sort: the idea

How most people sort a hand of cards:

- Keep the cards on the left **in order**.
- Take the next card, and **slide it left** past every larger card.
- Repeat until there are no cards left.



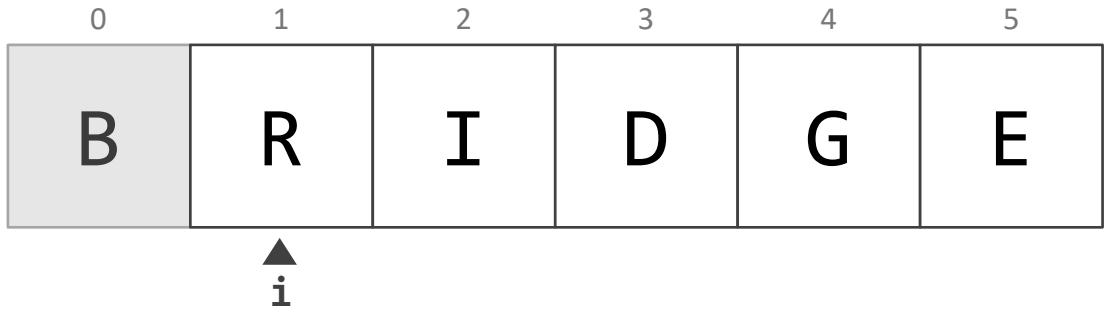
B I R are in order. **D** is next: it slides left past R and I, and stops at B.

Unlike selection sort, the gray part isn't final: items in it shift right to make room.

sorted so far (but not finished)

the item being inserted

Insertion sort: trace

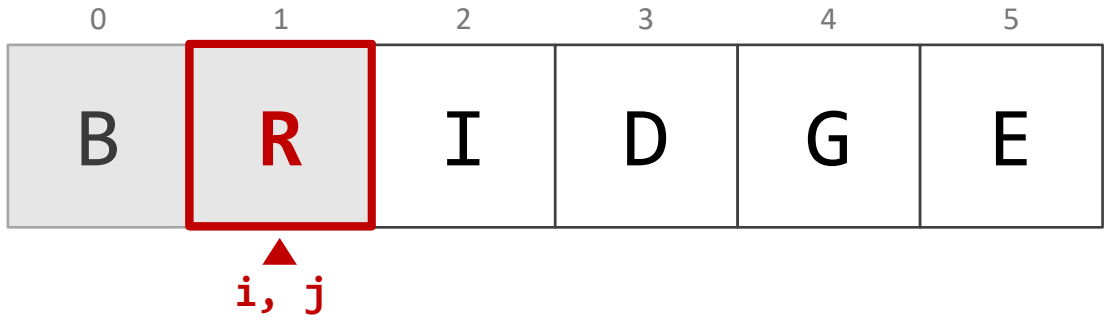


Start. `a[0..0]` is a single item, so it's already sorted. We start with **i = 1**.

sorted so far

the item being inserted

Insertion sort: trace

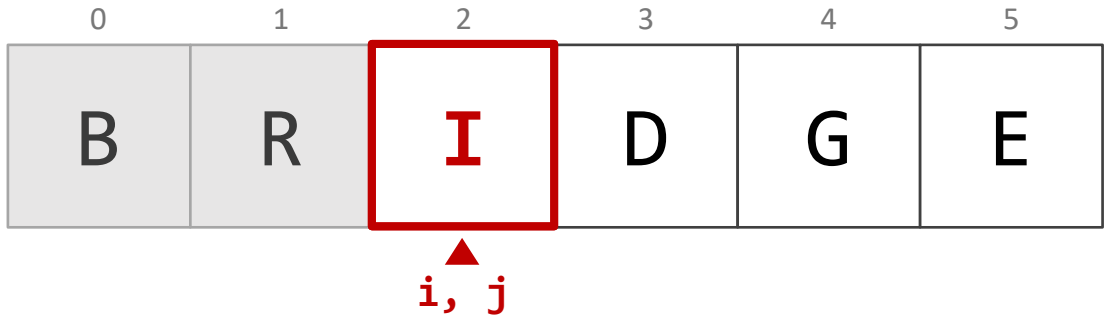


Pass i = 1. Insert **R**. Is R less than B? **No**, so R stops here. This pass made 0 exchanges, and `a[0..1]` is sorted.

sorted so far

the item being inserted

Insertion sort: trace

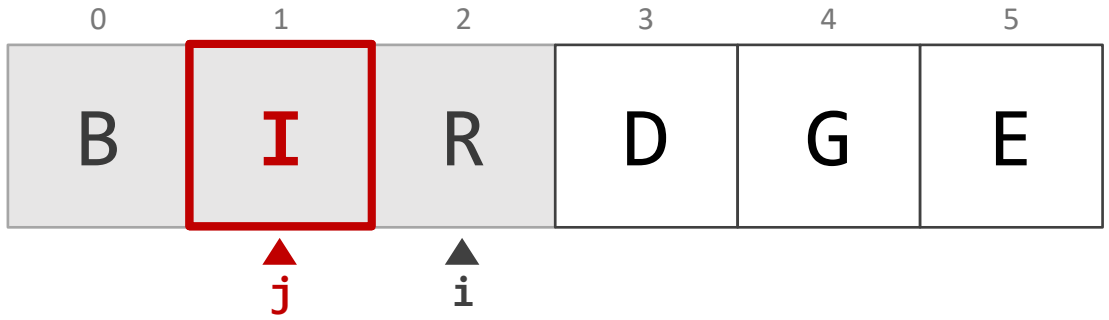


Pass i = 2. Insert **I**. Is I less than R? **Yes**: exchange them.

sorted so far

the item being inserted

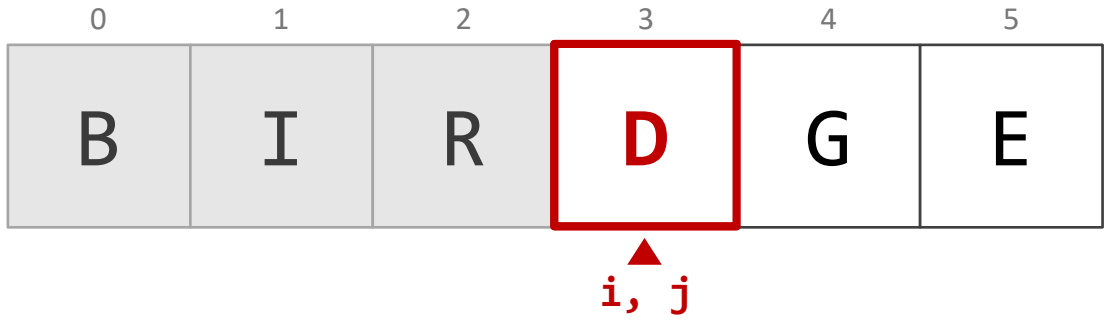
Insertion sort: trace



Is I less than B? **No**, so I stops here. This pass made 1 exchange, and `a[0..2]` is sorted.

sorted so far the item being inserted

Insertion sort: trace

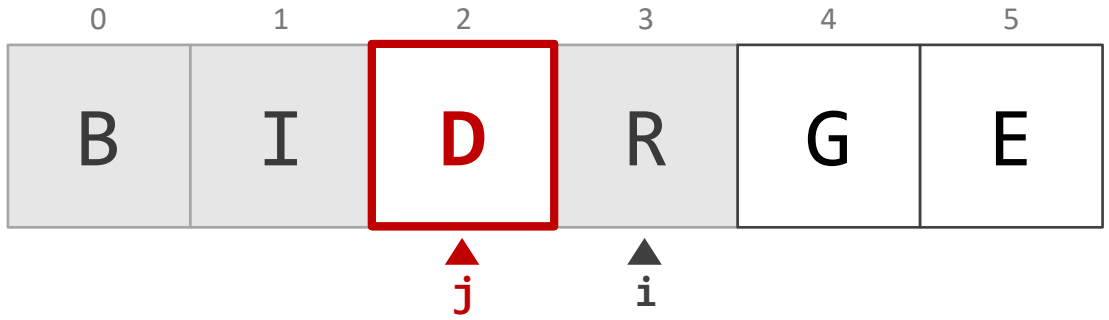


Pass i = 3. Insert **D**. Is D less than R? **Yes**: exchange them.

sorted so far

the item being inserted

Insertion sort: trace

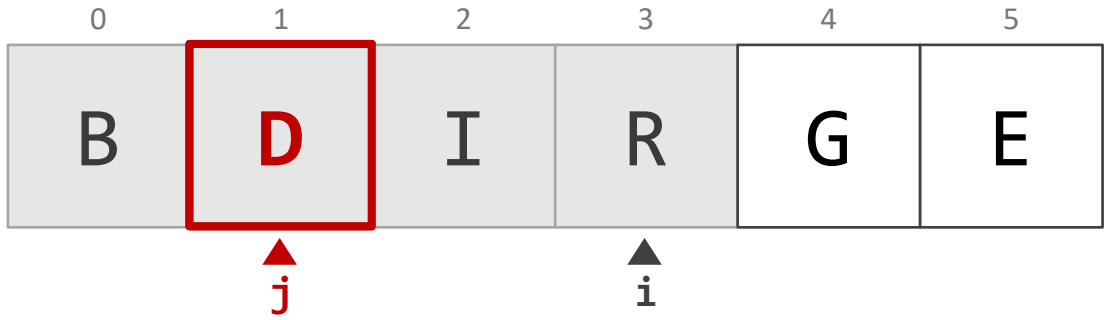


Is D less than I? **Yes**: exchange them.

sorted so far

the item being inserted

Insertion sort: trace

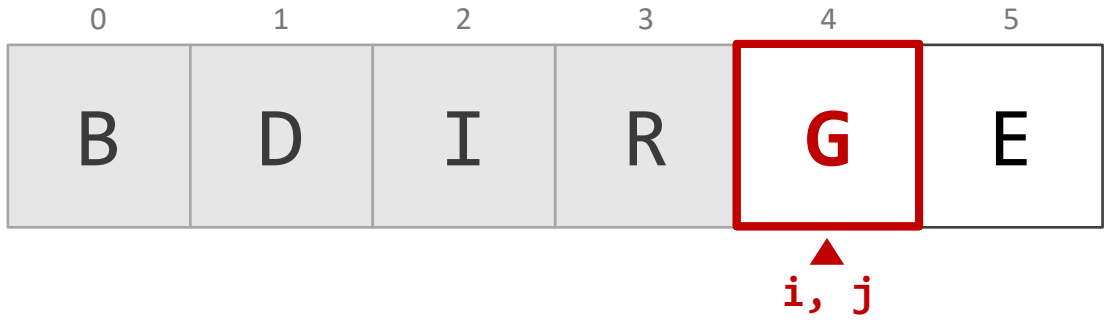


Is D less than B? **No**, so D stops here. This pass made 2 exchanges, and `a[0..3]` is sorted.

sorted so far

the item being inserted

Insertion sort: trace

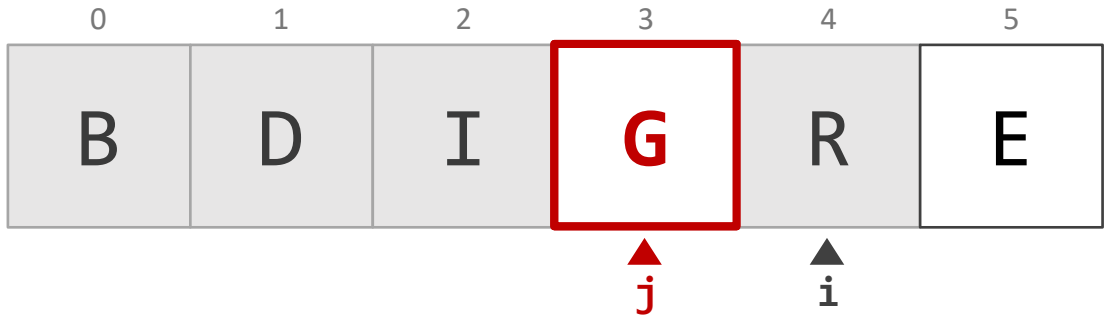


Pass i = 4. Insert **G**. Is G less than R? **Yes**: exchange them.

sorted so far

the item being inserted

Insertion sort: trace

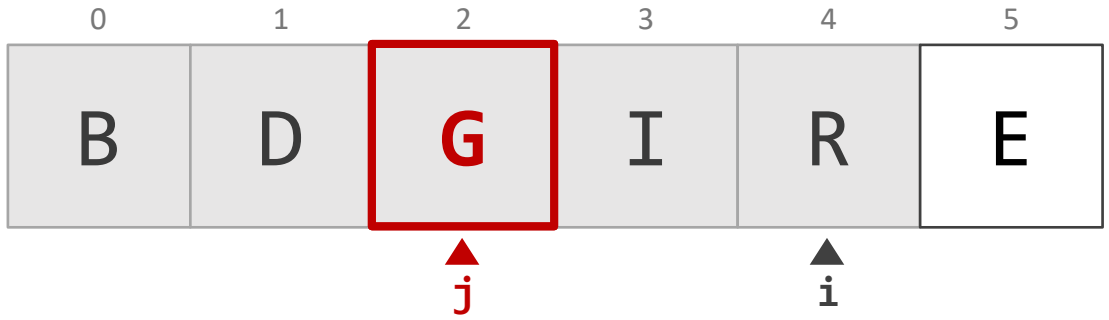


Is G less than I? **Yes**: exchange them.

sorted so far

the item being inserted

Insertion sort: trace



Is G less than D? **No**, so G stops here. This pass made 2 exchanges, and `a[0..4]` is sorted.

sorted so far

the item being inserted

Insertion sort: trace



Pass i = 5. Insert **E**. Is E less than R? **Yes**: exchange them.

sorted so far

the item being inserted

Insertion sort: trace

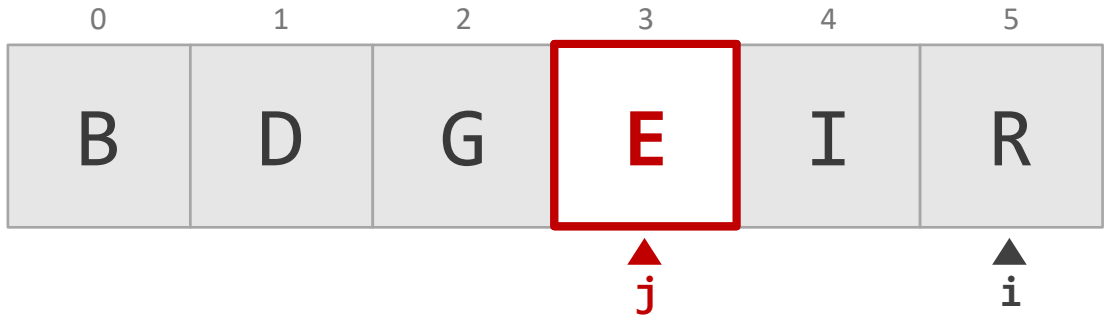


Is E less than I? **Yes**: exchange them.

sorted so far

the item being inserted

Insertion sort: trace

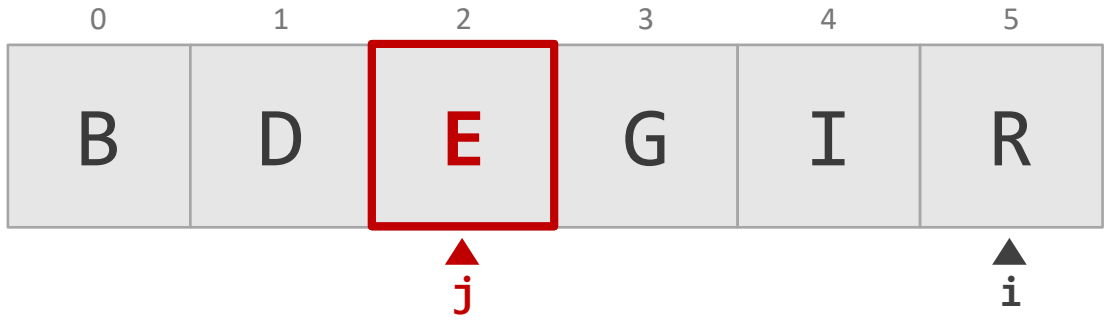


Is E less than G? **Yes**: exchange them.

sorted so far

the item being inserted

Insertion sort: trace



Is E less than D? **No**, so E stops here. This pass made 3 exchanges, and `a[0..5]` is sorted.

sorted so far

the item being inserted

Insertion sort: trace



Done. Five passes, **8 exchanges** in total. Hold on to that number.

Insertion sort: pseudocode

```
1  for i from 1 to n - 1:
2      j = i
3      while j > 0 and a[j] is less than a[j - 1]:
4          exchange a[j] and a[j - 1]
5          j = j - 1
```

The inner loop stops in one of two ways:

- the item to the left is **no larger** (it's in place), or
- `j` reaches **0** (it's the smallest so far).

This is the same pseudocode that's in the homework.

Pseudocode to Java: the careful spots

```
1  for i from 1 to n - 1:
2      j = i
3      while j > 0 and a[j] is less than a[j - 1]:
4          exchange a[j] and a[j - 1]
5          j = j - 1
```

1 line 1: "from 1 to n − 1" **includes** n − 1, so the Java condition is `i < n`. And n is `a.length`.

3 line 3: "is less than" is a call to `less(a[j], a[j - 1])`. Check `j > 0` **first**. When `j` is 0, `a[j - 1]` would be index −1, and Java crashes.

4 line 4: "exchange" is a call to `exch(a, j, j - 1)`.

Insertion sort: the input matters

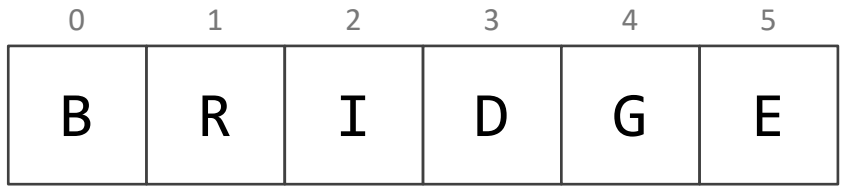
input	compares	exchanges
already sorted (best case)	~n	0
random (average case)	~$n^2/4$	~$n^2/4$
reverse sorted (worst case)	~$n^2/2$	~$n^2/2$

On sorted input, insertion sort is linear. Each item checks its left neighbor once, sees it's already in place, and stops.

That makes insertion sort great for arrays that are *almost* sorted. Selection sort can never do that.

Inversions

An **inversion** is a pair of items that are out of order relative to each other: the larger one comes first.



Inversions in **B R I D G E**:

R I R D R G R E I D I G I E G E

That's **8 inversions**. Insertion sort made **8 exchanges** on this array.

That isn't a coincidence. Explaining *why* is question 4 in your `trace.txt`. (Hint: what does one exchange of two neighbors that are out of order do to the number of inversions?)

Selection sort vs. insertion sort

	selection sort	**insertion sort**
compares, random input	~n²/2	~n²/4
compares, already sorted	~n²/2	~n
exchanges	n	~n²/4 on random input
notices sorted input?	no	yes

Both are **n^2** on random input: double n, and the time goes up about **4**×. You'll see exactly that in the homework's timing experiment.

For Homework

Part 1: translate the insertion sort pseudocode into Java in `Insertion.java`. Run `SortClient`, and look for OK.

Trace questions 1–4: insertion sort by hand on a different array. One line per pass, like this:

```
i = 1: [O, S, R, T, I, N, G]
i = 2: ...
```

If you get stuck, the homework page has an **"If you're stuck"** table for each part.

Next video: mergesort, which is much, much faster.